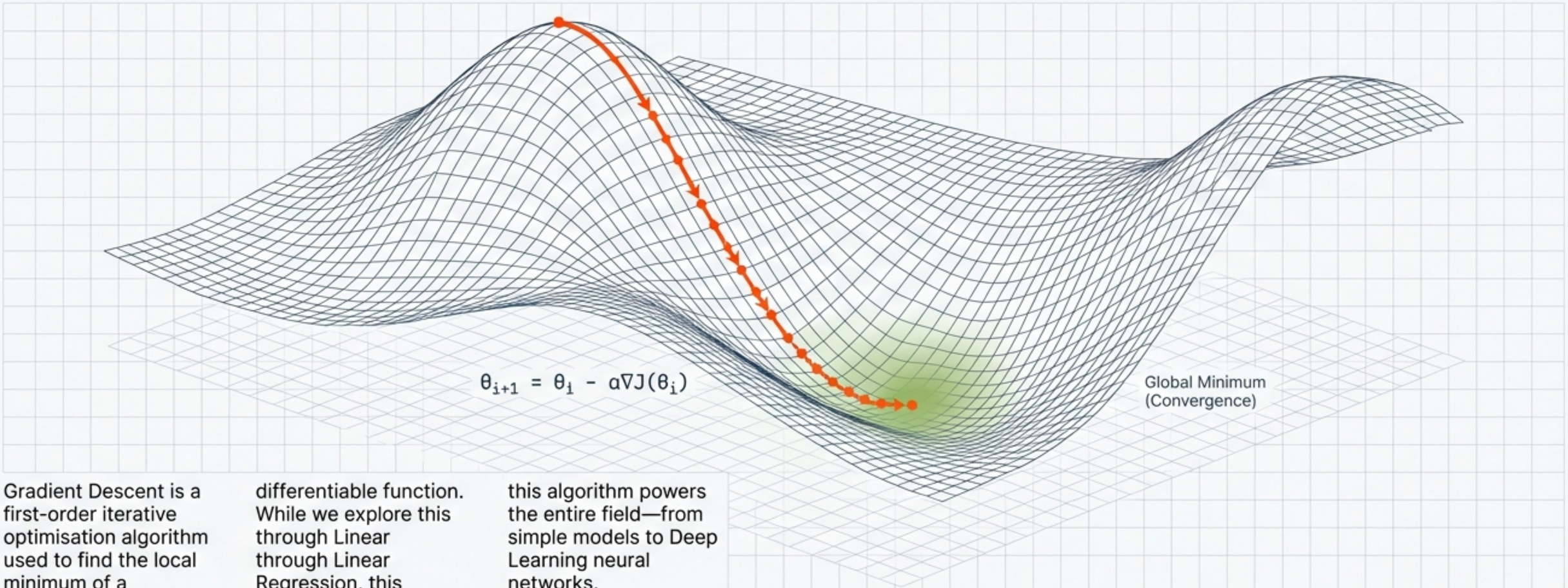
Gradient Descent:
The Engine of Machine Learning



Gradient Descent is a first-order iterative optimisation algorithm used to find the local minimum of a differentiable function. While we explore this through Linear through Linear Regression, this this algorithm powers the entire field—from simple models to Deep Learning neural networks.

```
for i in range(iterations):
    theta = theta - alpha * gradient(theta)
```

From Intuition to Calculus

The Computational Bottleneck of Analytical Solutions

The Analytical Approach

Ordinary Least Squares (OLS)

$$\theta = (X^T X)^{-1} X^T y$$

Matrix Inversion
Complexity ≈ O(n^3)

Calculating the perfect solution requires inverting a matrix. As data dimensionality increases, computational cost explodes.

The Iterative Approach

Gradient Descent

Iterative Approximation
Complexity ≈ O(kn)

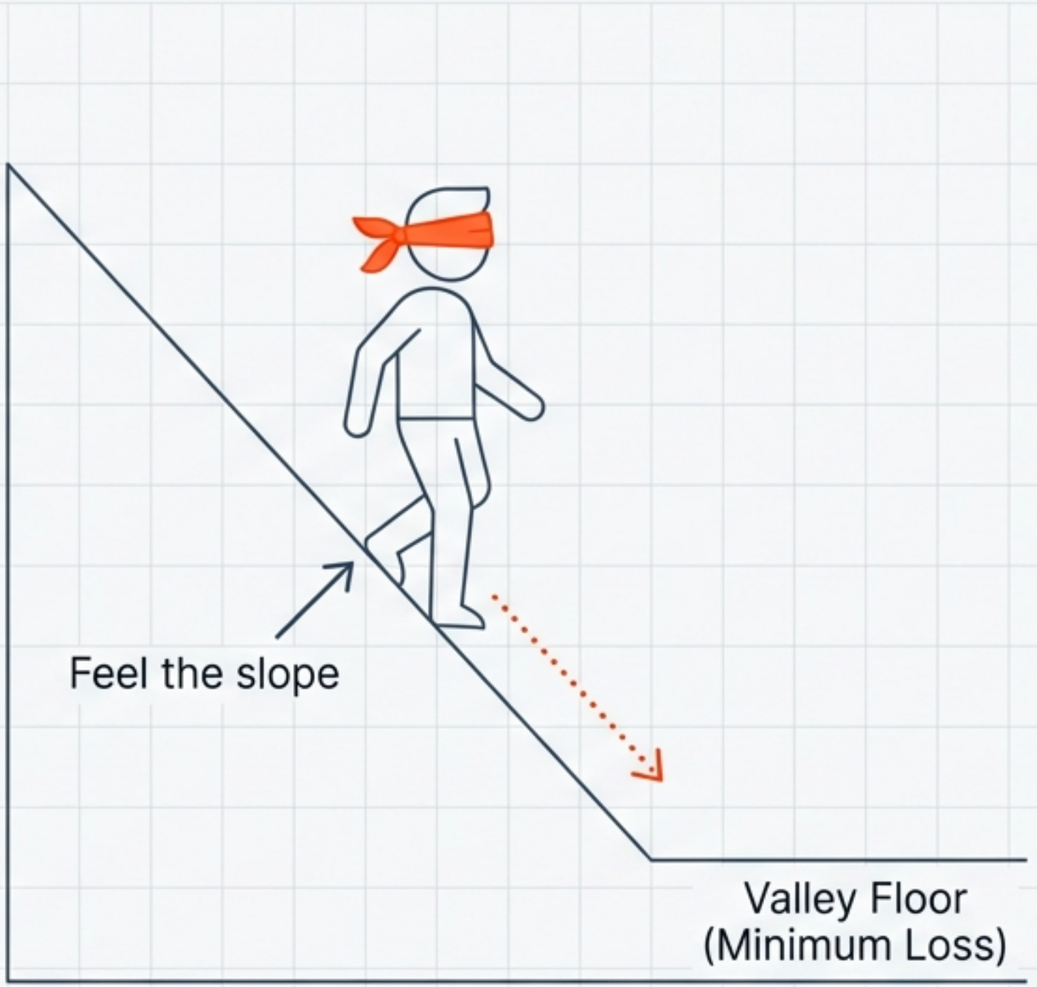
Instead of one massive calculation, we take small steps towards the answer. Scales efficiently with massive datasets.

Intuition: The Blindfolded Descent

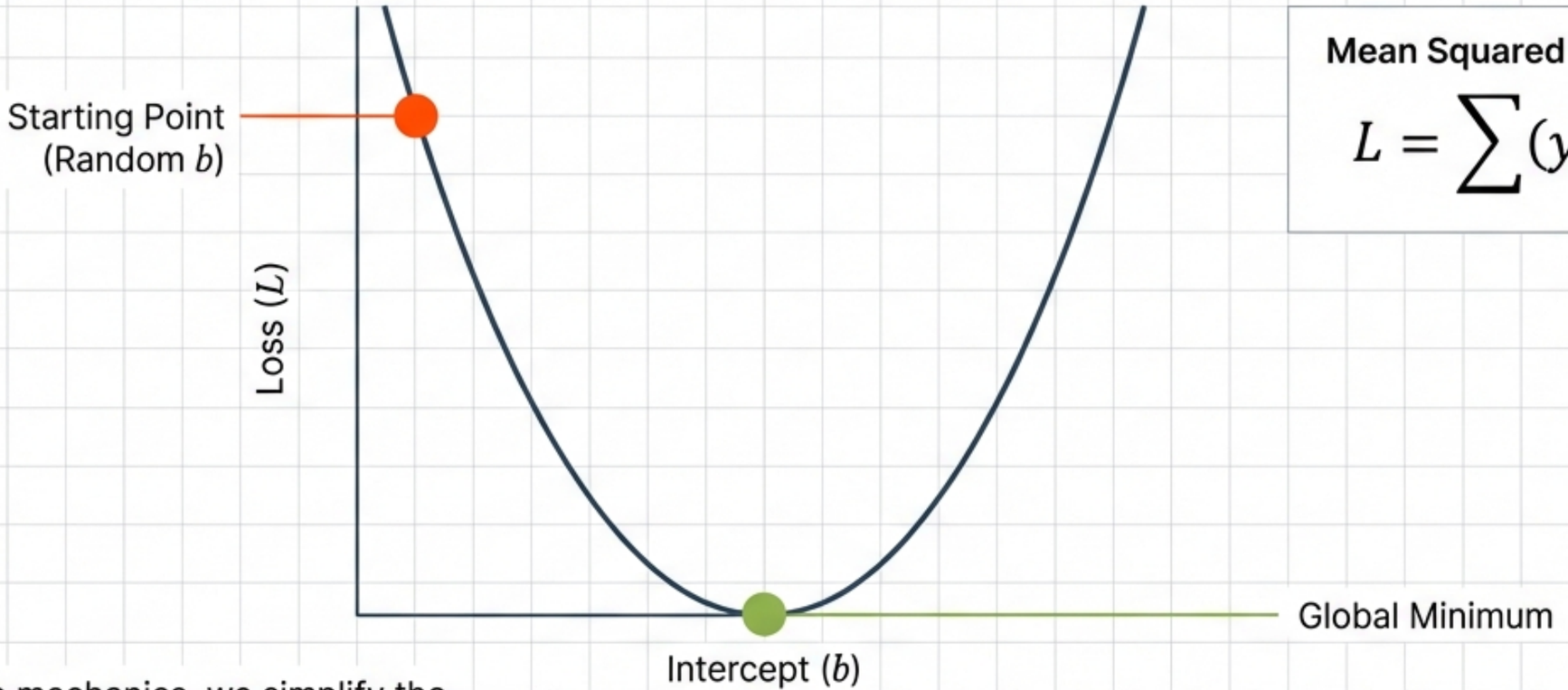
The Analogy: Imagine you are on a mountain, blindfolded. You need to reach the lowest point, but you cannot see the destination.

The Algorithm:

1. Feel the steepness under your feet (**Calculate Gradient**).
2. Take a step downhill (**Update Parameters**).
3. Repeat until the ground is flat (**Convergence**).



Defining the Landscape: The Loss Function

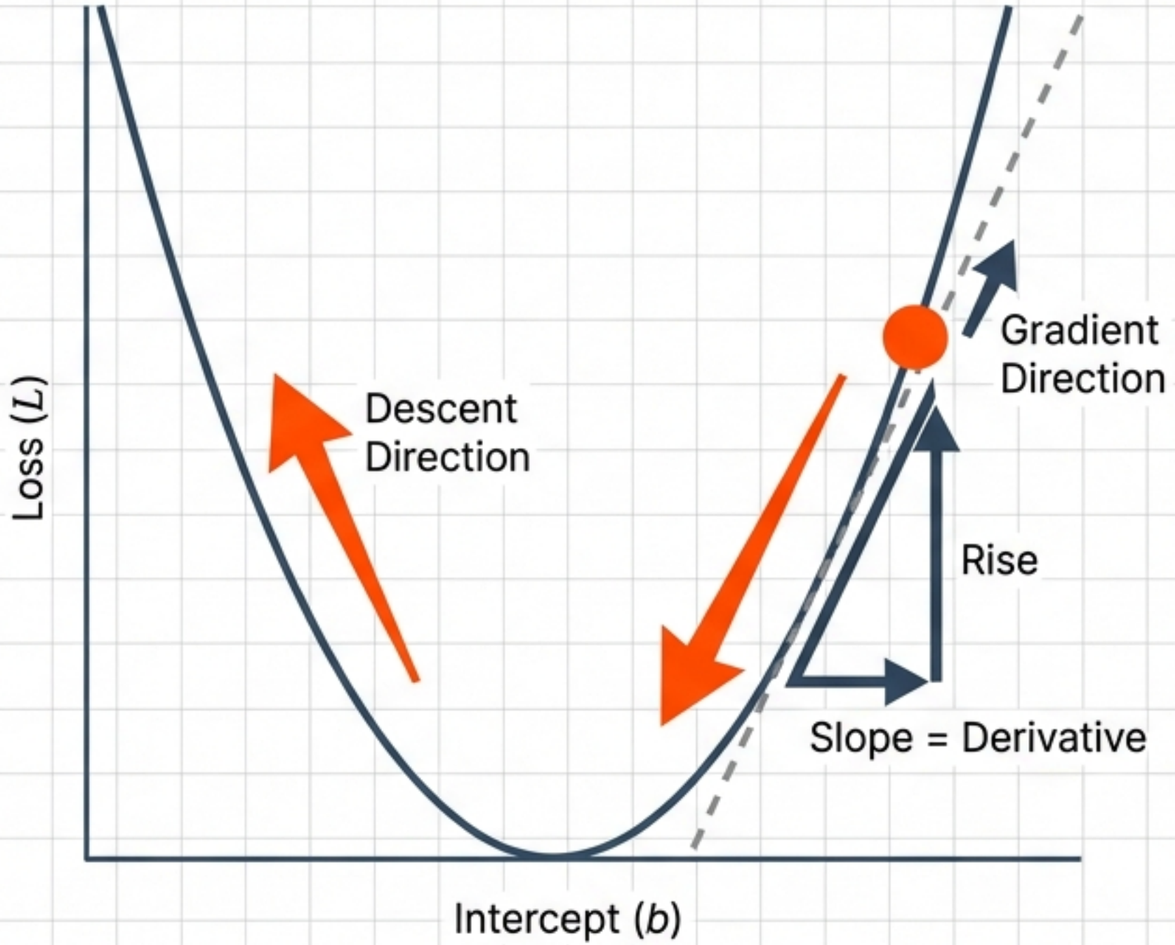


Mean Squared Error (MSE):

$$L = \sum (y_i - \hat{y}_i)^2$$

To understand the mechanics, we simplify the problem: we freeze the slope (m) and only optimise the intercept (b). This turns the complex 3D surface into a simple 2D parabola.

The Compass: Calculating Slope via Derivatives



The Math:

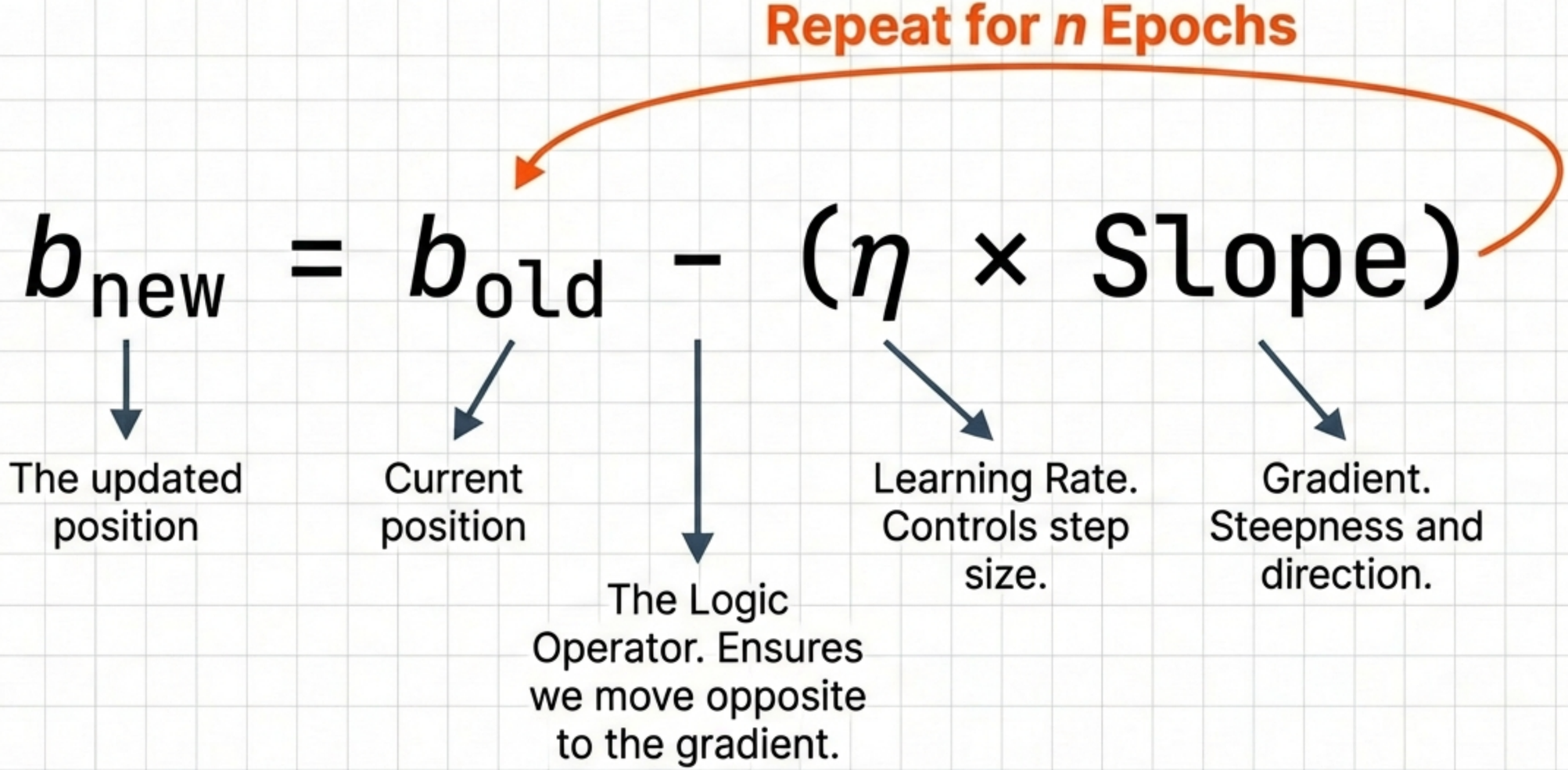
To 'feel the slope', we calculate the derivative of the Loss Function with respect to b: $\frac{\partial L}{\partial b}$.

The Logic:

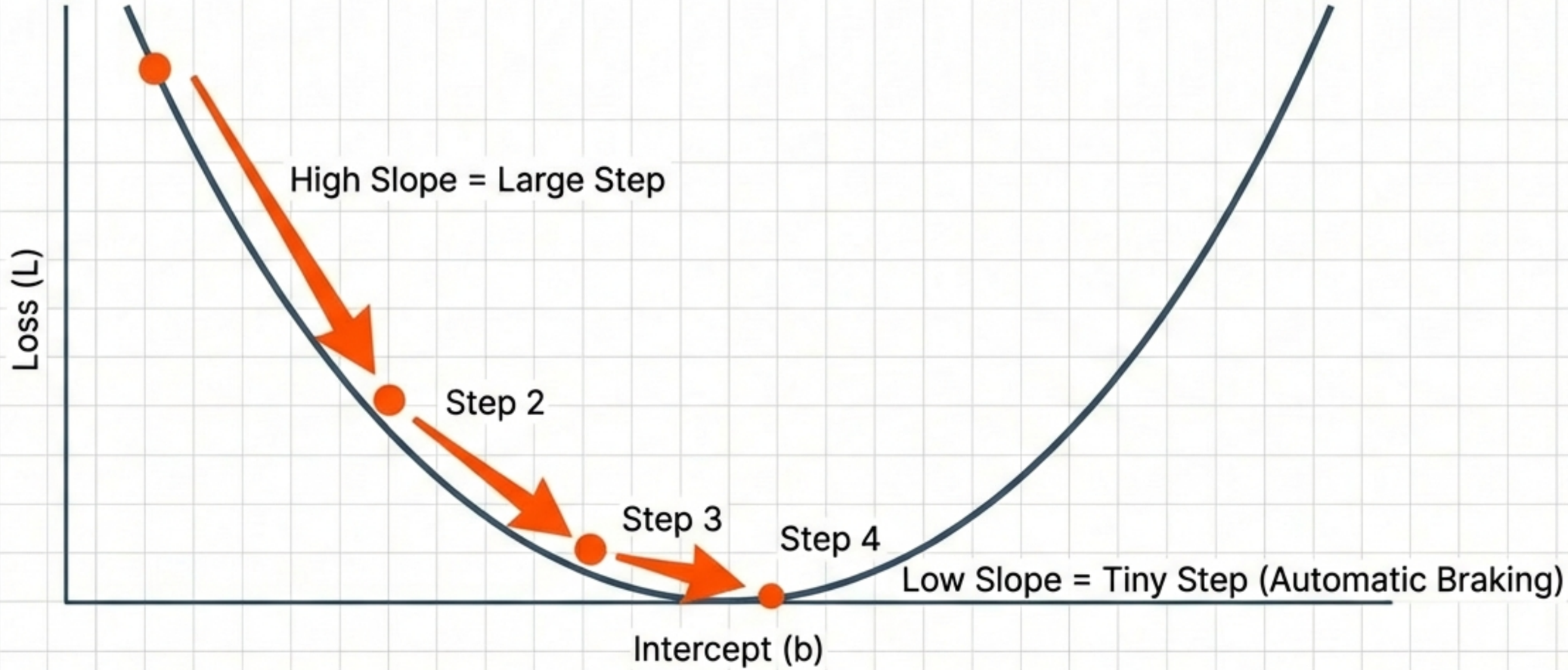
The derivative points *up* the mountain (ascent). To minimise loss, we must move in the opposite direction.

- **Positive Slope:** Move Left (Decrease b).
- **Negative Slope:** Move Right (Increase b).

The Engine: The Update Rule



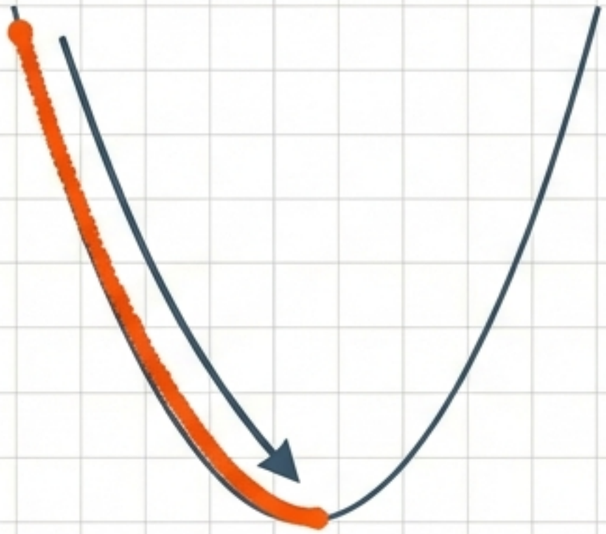
Visualising the Descent



The algorithm has built-in speed control. As the curve flattens (slope approaches 0), the step size naturally shrinks, allowing for a precise landing at the minimum.

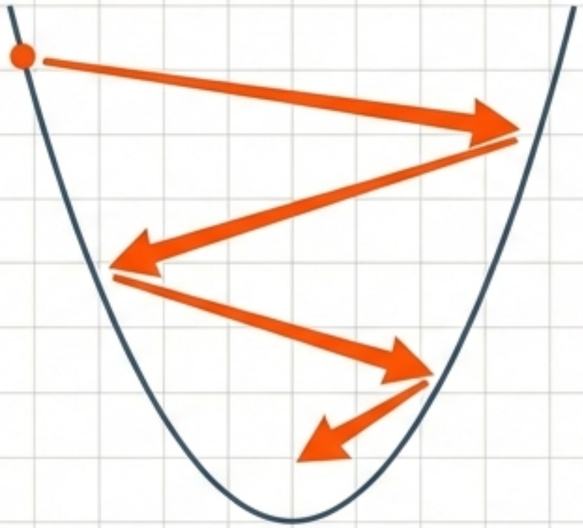
The Critical Parameter: Learning Rate (η)

Too Low, η = 0.0001



Inefficient / Slow

Too High, η = 1.0



Overshooting / Diverging

Just Right, η = 0.01

Converging

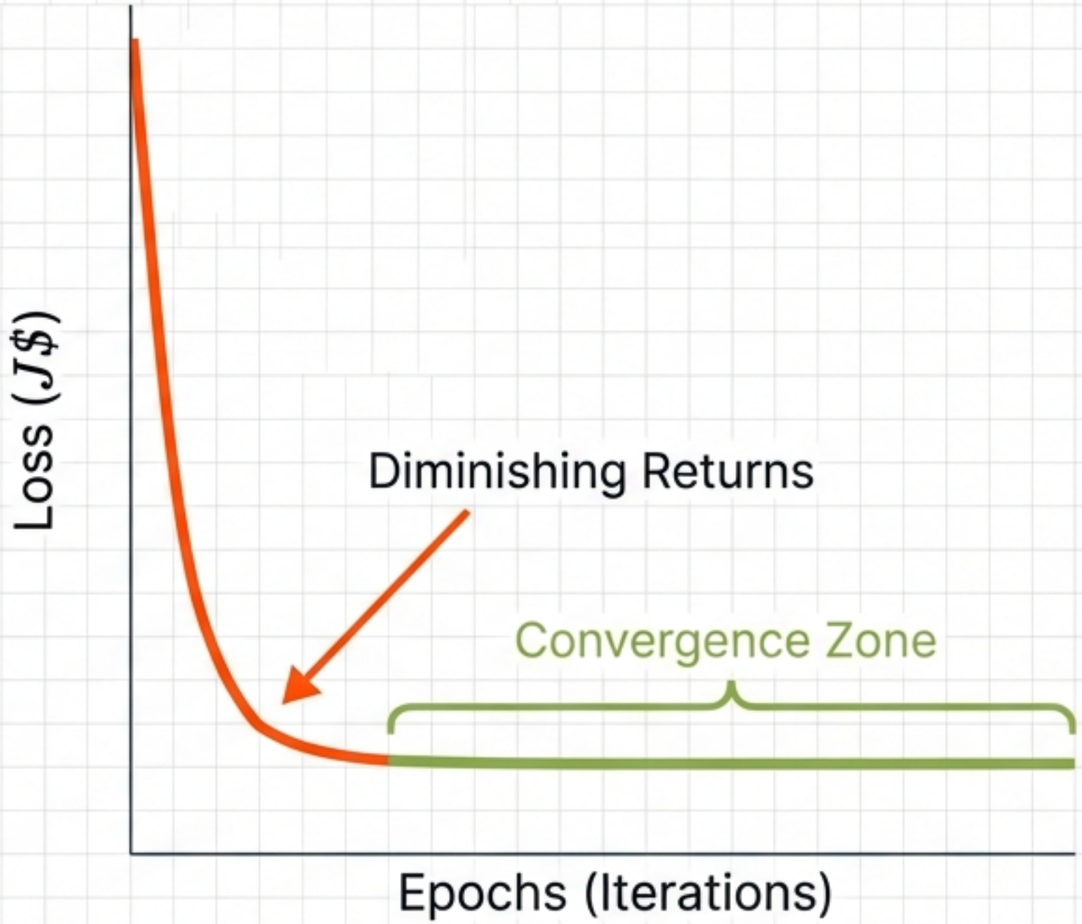
The **Learning Rate** is a hyperparameter that must be tuned. It dictates how aggressive the algorithm is.

Knowing When to Stop

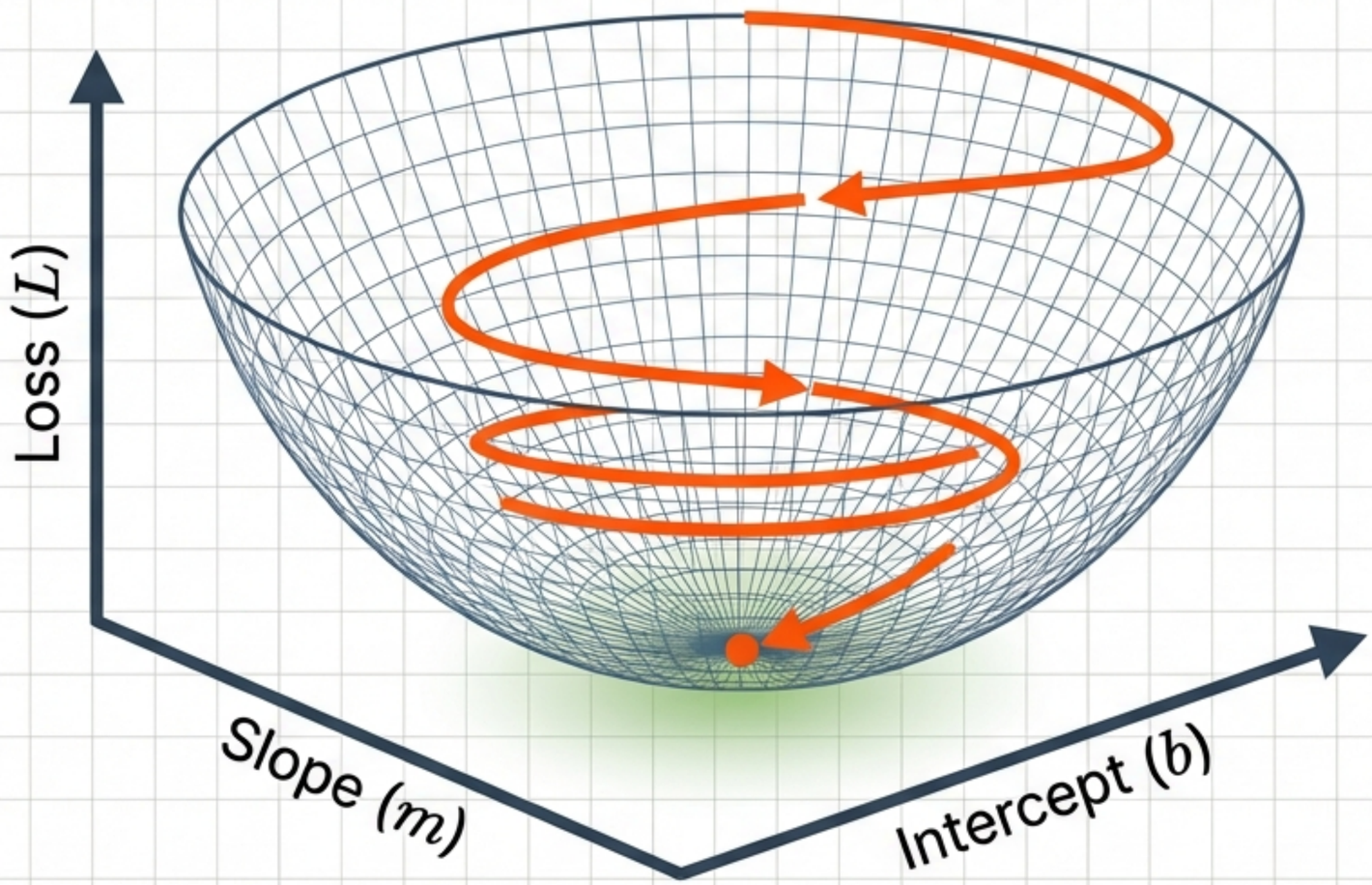
Stopping Condition 1: Max Epochs (e.g., 1000 loops).

Stopping Condition 2: Step Size Threshold. If $(b_{new} - b_{old}) < 0.001$, the model has converged.

Insight: Once the loss curve flattens, further computation yields negligible improvement.



Entering the 3D Realm: Multi-Variable Optimisation



We now unfreeze m. We are searching for coordinates (m, b) that correspond to the lowest point in this 3D landscape.
The “Blindfolded Hiker” analogy still holds, but now we can move 360 degrees.

From Slope to Gradient

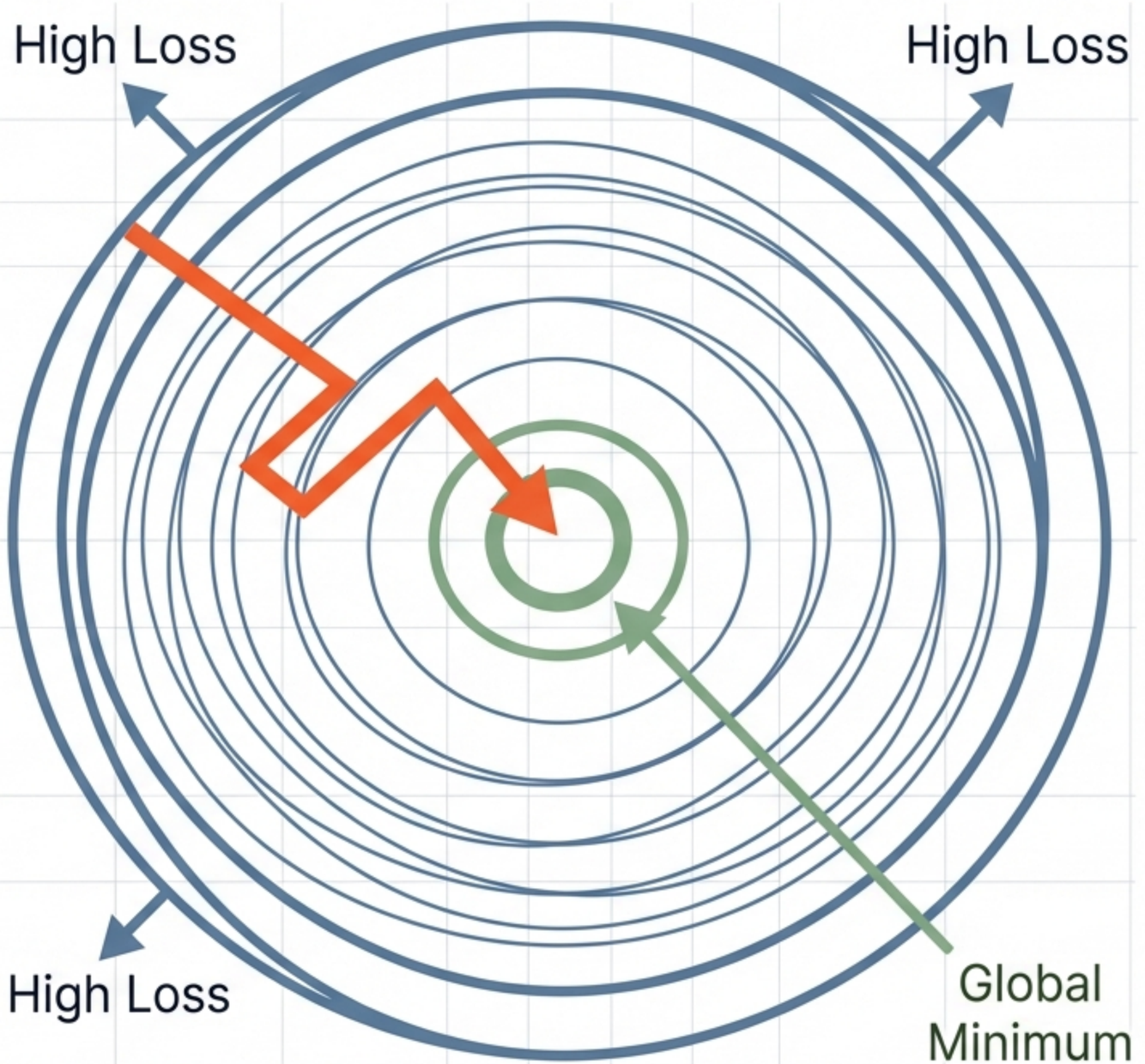
$$\frac{\partial L}{\partial b} = -2\sum(y - \hat{y})$$
$$\frac{\partial L}{\partial m} = -2\sum(y - \hat{y}) \cdot x$$

The Gradient Vector (∇J)

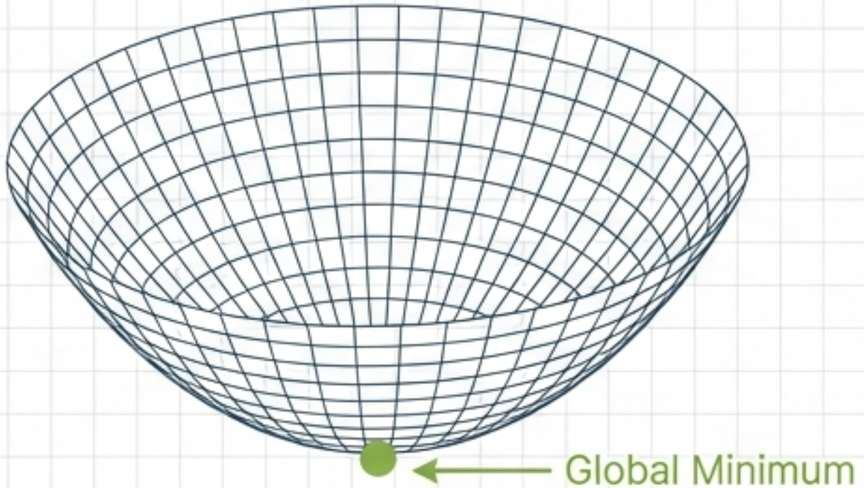
In multiple dimensions, the ‘slope’ becomes a vector called the **Gradient**. It contains the partial derivatives for every parameter. We calculate the steepness in the direction of *b* and the direction of *m* simultaneously, then update both.

Reading the Map: Contour Plots

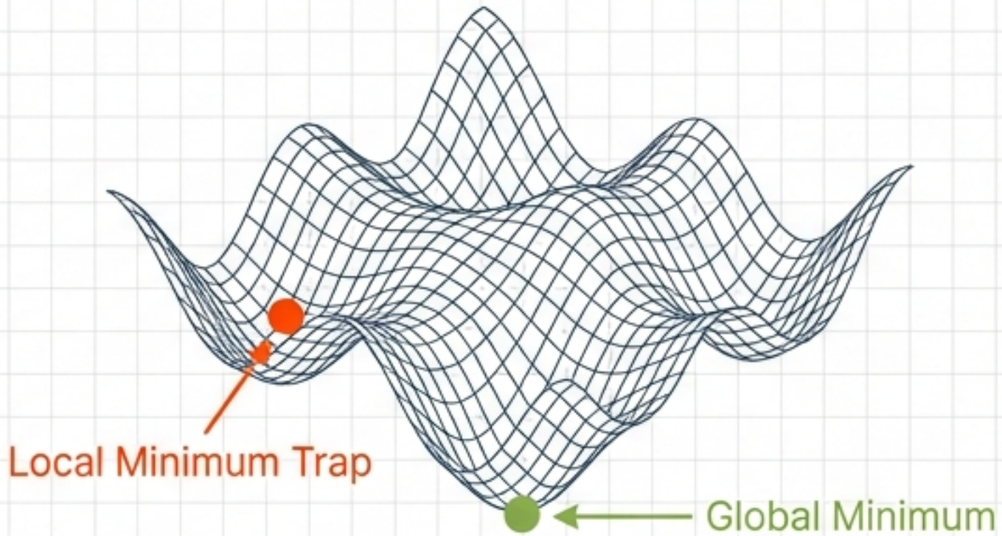
A Contour Plot is a top-down topographical map of the Loss Surface. The Gradient Descent algorithm always moves perpendicular to the contour lines, taking the mathematically steepest path towards the centre (the valley floor).



The Trap: Convex vs. Non-Convex Functions



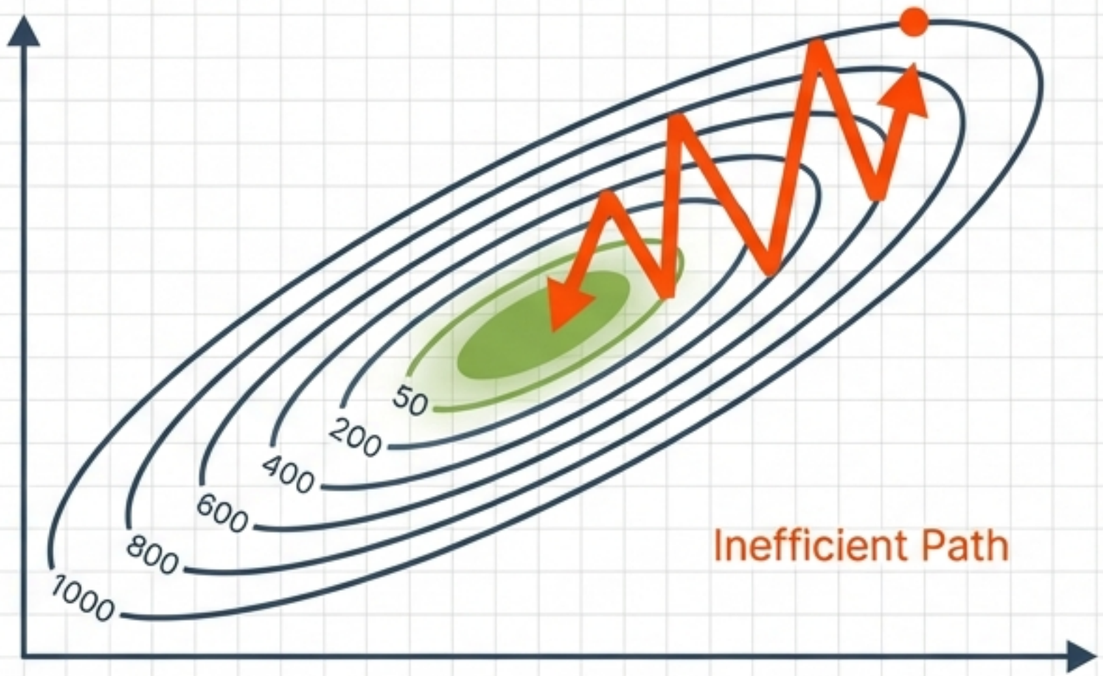
Convex - Linear Regression



Non-Convex - Deep Learning

Linear Regression creates a safe, convex bowl. Complex models create rugged landscapes where Gradient Descent can get stuck in shallow valleys (Local Minima) or flat plateaus (Saddle Points).

The Geometry of Data: Feature Scaling



If features have vastly different ranges (e.g., 0-1 vs 0-1000), the loss landscape becomes distorted. Standardising data creates a symmetrical bowl, ensuring fast and smooth convergence.

Universality: The Backbone of AI

Whether you are fitting a simple line through four data points or training a Neural Network to drive a car, the engine remains the same. The loss function changes, but Gradient Descent is always the navigator.